MobileAgents: Designing and Evaluating a Mobile Multimodal Interface for Controlling Personal AI Agents On the Go

Anonymous Author(s)

Abstract

People increasingly rely on personal AI agents—for research, messaging, scheduling—but controlling these agents away from a desktop is cumbersome: typing complex instructions on a phone is slow, and existing orchestration frameworks offer no end-user interface at all. We designed MobileAgents, a mobile-first platform where users bring their own AI agents and control them on the go through text, voice, and camera input. An LLM orchestrator decomposes multimodal requests into a dependency-aware plan across the user's personal agent pool, and a live DAG streams execution progress. The architecture is agent-agnostic: any agent can be registered with its role and tools at runtime—no code changes required. To study how much of this orchestration to reveal, we compared three transparency configurations—Black Box, Plan Preview, and Full Transparency—in a mixed-methods study with 10 participants (think-aloud, surveys, interviews). Plan Preview offered the best balance of trust and efficiency for mobile use; voice and image input were valued for on-the-go scenarios but required consent gating. We discuss value trade-offs, automation bias risks, and design implications for mobile multimodal agent control.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; **Empirical studies in HCI**.

Keywords

multimodal interaction, agent orchestration, transparency, human-in-the-loop, trust, human–AI interaction

ACM Reference Format:
Anonymous Author(s). 2026. MobileAgents: Designing and Evaluating a Mobile Multimodal Interface for Controlling Personal AI Agents On the Go. In . ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction: Use Case and Goals

People increasingly assemble personal toolkits of AI agents—one for paper search, another for drafting, another for team messaging—but controlling these agents is tethered to the desktop. Typing multi-step instructions into a terminal or notebook is impractical when walking between meetings, commuting, or working from a phone. Current orchestration frameworks (CrewAI, LangGraph, AutoGen) compound this problem: they are developer-facing APIs with no end-user interface at all, let alone a mobile one. The demand for mobile agent control is real: Clawdbot [**?**], an open-source autonomous AI agent that operates via messaging platforms (WhatsApp, Telegram, Slack), grew to an estimated 300,000–400,000 users within months of its 2025 launch by letting people delegate tasks—reading emails, managing calendars, deploying code—from their phone. Yet Clawdbot operates as a single opaque agent with broad permissions and no plan preview, approval gates, or transparency into what it will do before it acts, drawing scrutiny from security researchers over misconfigured instances accessing sensitive services. AI systems always involve value-laden choices, even when they appear purely technical [19]; orchestration platforms are no exception, yet they offer no mechanism for users to inspect plans, intervene before irreversible actions, or issue commands via voice or camera. Shneiderman [21] argues that human-centred AI must be simultaneously reliable, safe, and trustworthy. Amershi et al. [1] distil 18 guidelines for human–AI interaction, including "make clear what the system can do" (G1) and "support efficient correction" (G2); agent orchestration platforms violate nearly all of them by design, since they have no user-facing layer at all.

We designed **MobileAgents**, a mobile-first platform that lets users *bring their own AI agents* and control them on the go through text, voice, and image input. The architecture is **agent-agnostic**: any agent can be registered at runtime by defining its role, tools, and metadata—no code changes to the orchestration or UI layer. An LLM orchestrator decomposes multimodal requests into a dependency-aware execution plan across the user's personal agent pool, and a live directed acyclic graph (DAG) streams progress in real time. The platform ships with five demo agents (ArXiv, Proposal, Wikipedia, Slack, Semantic Scholar), but these are illustrative; the value proposition is that users plug in *whatever agents matter to them* and manage them from their phone through whichever modality suits the moment—voice while walking, a photo of a whiteboard in a meeting, or text at a desk.

A multi-agent orchestrator is a particularly challenging AI *design material*: its behaviour is emergent (plans depend on which agents are registered and what the LLM decides), data-dependent (multimodal inputs shape plans unpredictably), and opaque (multiple agents execute in parallel behind the scenes). Recent scaling studies show that multi-agent coordination amplifies errors—centralized architectures amplify errors 4.4× compared to 17.2× for independent agents [12]—motivating our centralized orchestrator design. Capel and Brereton [5] map the HCAI research landscape and identify transparency, user agency, and contextual awareness as under-explored in agentic systems. The central design question is therefore *how much* of this internal orchestration to expose on a small screen. We operationalise this as three configurable **transparency modes**—Black Box, Plan Preview, and Full Transparency—and evaluate their impact on users.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Stakeholders and values. We adopted the tripartite VSD methodology [7, 8]—conceptual, empirical, and technical investigations—as our overarching design framework. In the *conceptual investigation*, we identified direct and indirect stakeholders following Friedman et al.'s [7] stakeholder analysis method and performed a *value source analysis* to distinguish project-endorsed values (usability, performance) from stakeholder-held values that might diverge from designer assumptions [7]. Primary stakeholders: researchers and professionals who maintain personal agent pools and need mobile access. Secondary: collaborators who receive agent-generated outputs (e.g., Slack messages sent on the user's behalf). Tertiary: IT administrators concerned with data governance when agents act across services. We then conducted an *empirical investigation* using value-oriented semi-structured interviews [7]: informal conversations with four graduate students, structured as participatory inquiry. Participatory design, rooted in Scandinavian workplace democracy movements of the 1970s, holds that people affected by a technology should have a voice in shaping it [20]—not merely as consultees but as genuine collaborators. This co-design orientation ensured that values emerged from lived experience rather than designer assumptions. Three core values surfaced: **V1 Efficiency** ("I want to fire off a request from my phone and get a result—don't slow me down"), **V2 Control** ("I need to verify before it sends anything on my behalf"), and **V3 Privacy** ("I don't want my voice recorded or images stored"). These values are in tension—V1 favours opacity, V2 demands visibility, V3 constrains which modalities feel safe in public—a pattern Friedman et al. [7] call *value tensions*, which drive design trade-offs rather than admit simple resolution. This motivates our research question.

2 Methods: System Design and Implementation

2.1 Prototype Overview

MobileAgents is a fully functional prototype (Figure 1; full architecture in Appendix A–B). The client is built in React with Konsta UI for mobile components as a Progressive Web App. The backend is a FastAPI server and communicates with the client through REST APIs and Server-Sent Events. The Orchestrator and demo agents are implemented with CrewAI and powered by GPT-4.1.

Users interact with three modalities optimized for mobile that can be configured in settings (Figure 2). **Multimodal** supports text, images, and speech, and responds with text and visualizations, with an optional text-to-speech button. **Text and Image** mode supports text and images as input and responds with text and visualizations. **Voice-only** mode is unique as it supports only speech input and responds only in speech. While this enables convenience, it also restricts visualization, planning, and approval, and is designed to be more concise. The orchestrator decomposes the request into a dependency graph of tasks and assigns them to the corresponding agents. Responses are streamed in real-time updates via Server-Sent Events.

The three execution control settings are **Black Box**, **Planning**, and **Full Transparency** (Figure 3). Black Box enables the orchestrator and agents to execute tasks autonomously. Planning proposes a plan that requires approval. Full Transparency proposes a plan that requires approval and embeds an execution graph that updates in real time as the multi-agent system executes tasks, providing full visibility into the system's operation. Transparency into agents is available through the agent tab, where users can view each agent, their available tools, and system prompts.

Additional control is given to the user through agentic settings. The Orchestrator takes in a Constitution inspired by Anthropic's Constitutions [**?**]. This constitution provides instructions to guide orchestration logic, planning, and response stylization. It is editable from the UI and does not require system updates. Furthermore, individual agents can be turned on or off, allowing users to control which agents are accessible to the orchestrator. This functionality works in tandem with the execution control settings discussed before to enable users to manage risk. For example, a high-risk agent can be deployed with full transparency mode, and a low-risk one can be used effectively in Black Box.

2.2 Design Rationale

The *technical investigation* [7, 8] translated values into design requirements. We drew on Amershi et al.'s [1] 18 HAI guidelines, Shneiderman's [21] HCAI framework, and van de Poel's [25] value-to-requirement hierarchy. Umbrello and van de Poel [24] demonstrate that VSD can be systematically mapped onto AI systems by treating values as first-class design constraints; Sadek et al. [17] provide concrete guidelines for integrating VSD into responsible AI toolkits, recommending that value–design mappings be documented and traceable. We followed this approach, mapping each stakeholder value to a concrete design implication and tracing it to specific HAI guidelines:

- **V1→D1:** Black Box mode for speed-oriented users (no plan, no graph, auto-execute). This prioritises Amershi et al.'s G5 ("match relevant social norms") for users who expect instant results on mobile.
- **V2→D2:** Plan Preview and Full Transparency modes with approval gates before irreversible actions. This operationalises G1 ("make clear what the system can do"), G2 ("support efficient correction"), and G11 ("make clear why the system did what it did") [1]. Jerry et al. [10] argue for *human oversight-by-design*: embedding human judgment throughout the AI pipeline via risk detection and escalation, rather than appending it as a final review step. Our checkpoint gates realise this principle—they are distributed throughout the execution graph, not just at the end. Shneiderman's [21] audit trail principle is realised through the live DAG, which provides a visual record of every agent action.
- **V3→D3:** Consent-gated multimodality—camera and microphone trigger a privacy dialog on first use explaining that data is sent to OpenAI; consent is revocable. This addresses G16 ("convey the consequences of user actions") [1] and Shneiderman's safety requirement.

2.3 Persona, Tone, and Agent Customisation

We designed an AI orchestrator persona (full text in Appendix D): a concise research librarian that uses uncertainty-aware language (addressing Amershi et al.'s G13: "mitigate social biases" [1] by avoiding overconfident claims), prefers parallel execution, and never claims authority over institutional policy. Critically, the orchestrator's system prompt includes a user-editable "constitution" field—free-text

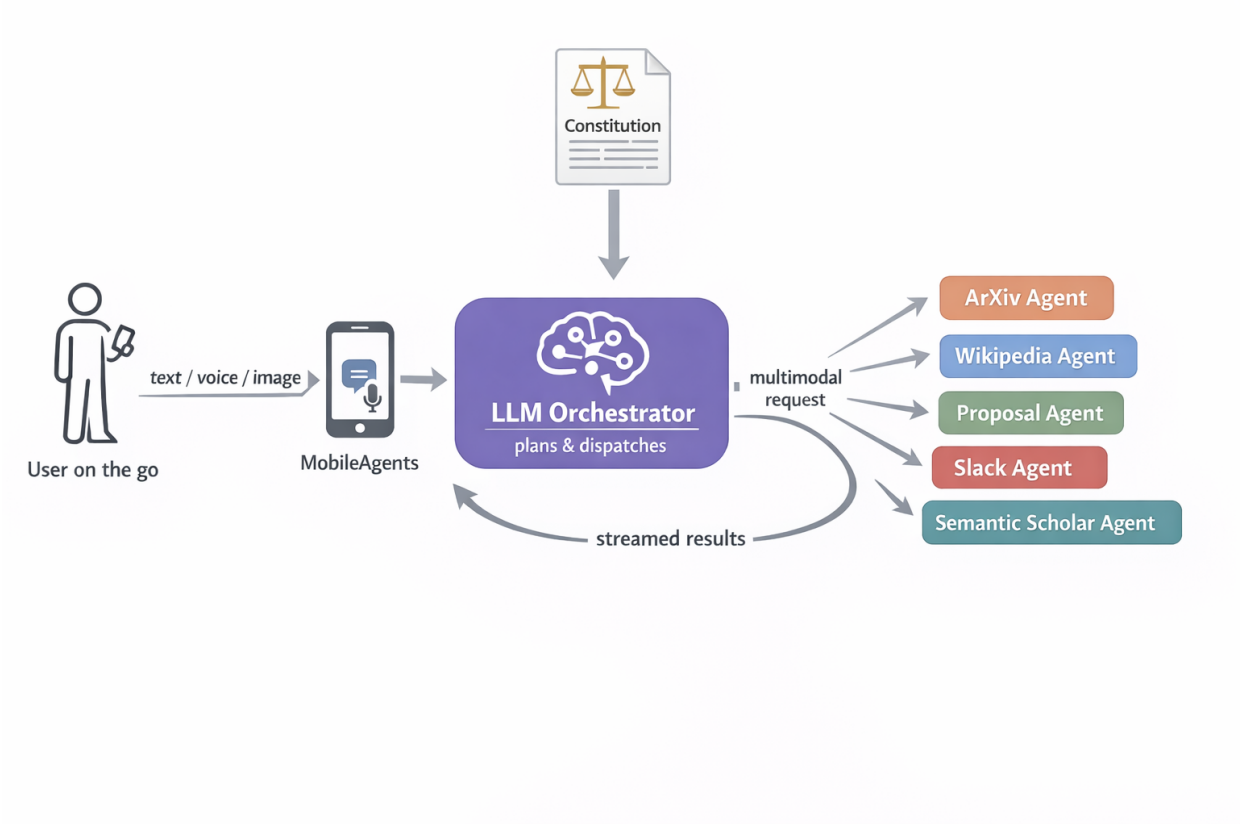


Figure 1: MobileAgents system overview. A user on the go issues a multimodal request (text, voice, or image) from their phone. The LLM Orchestrator decomposes it into a dependency-aware plan and dispatches tasks across the user's personal agent pool (ArXiv, Wikipedia, Proposal, Slack, Semantic Scholar). Results stream back to the mobile client in real time.

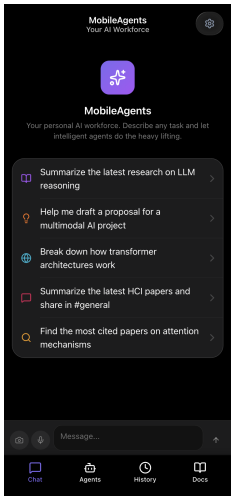
guidelines that are appended at planning time. This lets users shape *how* their personal agent pool is coordinated (e.g., "always run ArXiv and Semantic Scholar in parallel," "never send Slack messages without showing me a draft first"). The constitution operationalises co-design's core tenets [20]: empowerment (giving users control over decisions that affect them), mutual learning (users discover orchestration possibilities while the system learns user preferences), and democratisation of design (opening up AI coordination rules that are normally developer-only). Sanders and Stappers [20] call this moving users from "passive consumer" to "co-creator." It also operationalises Amershi et al.'s G15 ("provide global controls") [1] and Shneiderman's [21] principle that users should be able to set automation boundaries. Each worker agent can be enabled, disabled, inspected, or deleted from the Agents tab, giving users full control over which agents are available for orchestration. Sadek et al.'s [16] value-sensitive conversational agent co-design framework demonstrates that co-designing agent values with end users yields more contextually appropriate behaviour; their comprehensive review of co-design methods for conversational agents [15] further recommends embedding value elicitation throughout iterative design cycles rather than treating it as a one-off activity. Our constitution mechanism extends this idea by making value alignment an ongoing, user-driven activity embedded in the runtime system itself. This also embodies VSD's principle of *co-evolution of technology and social structure* [7]: as users write new constitution rules, both the technology's behaviour and the social practices around it evolve together.

2.4 Execution Control

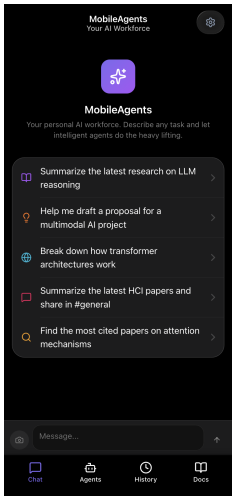
Our independent variable is **transparency level**, operationalised as three configurations (Table 1):

Table 1: The three transparency configurations compared.

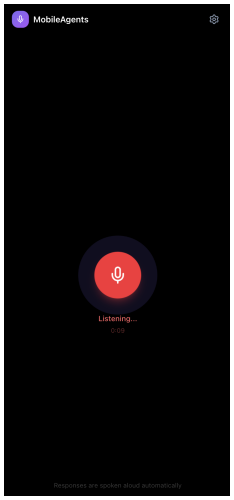
Configuration	What the user sees and can do
C1: Black Box	Query in, final answer out. No plan card, no graph, no approval step. Agents auto-execute.
C2: Plan Preview	Orchestrator's task plan shown as a card. User approves with one tap. A spinner indicates execution (no live graph).
C3: Full Transparency	Plan card *plus* a live DAG graph with animated node/edge status, step-by-step results streaming in real time, and checkpoint approval gates for sensitive actions.



(a) Multimodal mode

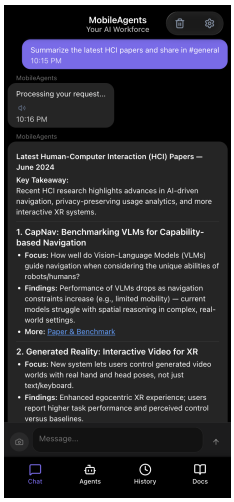


(b) Text + Images mode

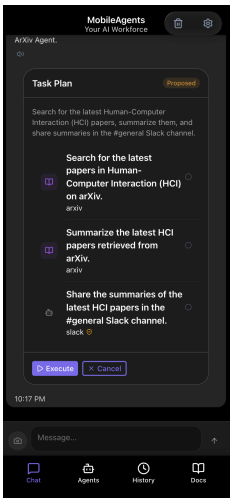


(c) Voice-only mode (listening)

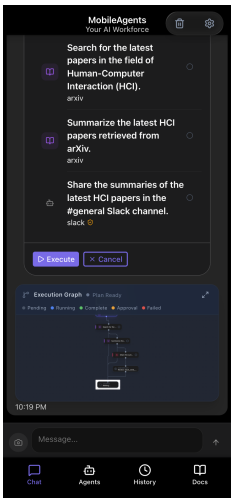
Figure 2: MobileAgents input modes. (a) Multimodal: text, camera, and microphone all active. (b) Text + Images: camera available but voice disabled. (c) Voice-only: full-screen recording interface; responses are played back via TTS. All three modes unify into the same planning prompt.



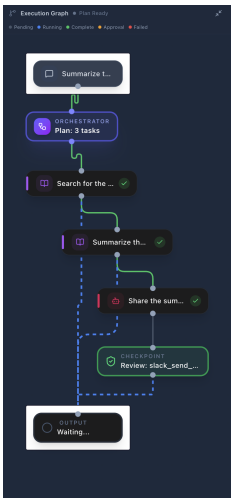
(a) C1: Black Box



(b) C2: Plan Preview



(c) C3: Full Transparency



(d) Live DAG (C3 detail)

Figure 3: The three transparency configurations and the C3 execution DAG. (a) Black Box: final answer only, no plan or graph. (b) Plan Preview: task plan card with Execute/Cancel before agents run. (c) Full Transparency: plan card plus live DAG. (d) DAG close-up showing orchestrator, agent step nodes, animated edges, and a checkpoint gate before the Slack send step.

These configurations operationalise Shneiderman's [21] two-dimensional framework: C1 sits at high automation / low human control, C3 at high automation / high human control, and C2 in between. This lets us test the trade-off between V1 (efficiency) and V2 (control) while keeping modality support constant across conditions.

2.5 Accessibility and Inclusion

HCAI demands inclusivity across age, language, culture, and ability levels [5]. MobileAgents supports English, Chinese, Arabic with full RTL layout, and Bulgarian. It has dark, light, and auto color themes. Icon buttons contain ARIA labels. Users with different abilities and devices can utilize the product through multimodal inputs of text, voice, and images, with neither being a point-of-failure. The design was chosen with personalization in mind and to not impose a single interaction pattern.

3 Methods: Research Question, Evaluation Methods, and Metrics

3.1 Research Question

RQ: How does the level of transparency in a multi-agent AI orchestrator affect user trust, perceived control, and task efficiency?

3.2 Study Design

We conducted a within-subjects study: each participant used all three configurations in counterbalanced order (Latin square) to control for learning effects. To simulate realistic mobile, on-the-go scenarios, participants completed tasks using different modalities: (T1) type a text query to find and compare two recent papers on a given topic, (T2) photograph a printed paper abstract with the camera and ask the system to find related work and draft a proposal, and (T3) use voice input to dictate a request to summarise results and send them to a Slack channel. These tasks exercise the full multimodal range and span the agent pool (ArXiv, Wikipedia, Proposal, Slack). Tasks were matched in complexity and counterbalanced across configurations.

3.3 Participants

We recruited 10 participants (6 male, 4 female; ages 22–28, $M = 24.3$) from a university graduate programme. All had experience with LLM-based tools but none had used multi-agent systems. Participation was voluntary.

3.4 Evaluation Methods

Traditional usability metrics (task completion, click-through rates) are insufficient for AI systems, where evaluation must also capture trust calibration, mental model accuracy, perceived agency, and error tolerance [4]. We therefore used a mixed-methods approach—essential for capturing the full AI experience—with three methods, all directly involving users:

(1) **Think-aloud sessions** (user-involving, qualitative): Participants verbalised their thoughts while completing tasks. Sessions were screen-recorded and the researcher took timestamped notes on hesitations, errors, and interaction breakdowns. This method surfaces *in-situ* mental model accuracy and trust calibration [4]—whether users' expectations of what the system will do match what it actually does—which surveys alone cannot capture.
(2) **Post-task survey** (user-involving, quantitative): After each configuration, participants rated 7-point Likert items for *trust*, *perceived control*, and *satisfaction* (full items in Appendix M). Trust items draw on Jian et al.'s [11] empirically validated Trust in Automation scale, which measures trust as a function of perceived reliability and predictability. Usability items draw on Brooke's [3] System Usability Scale. These constructs are well suited because our research question centres on how transparency mediates trust and control.
(3) **Semi-structured interview** (user-involving, qualitative): After all three conditions, a 10-minute interview explored preferences, trust calibration, privacy concerns, and suggestions (questions in Appendix N). Interview data was analysed alongside think-aloud transcripts.

3.5 Metrics

(1) **Trust, perceived control, and satisfaction** (Likert, 1–7): from the post-task survey, grounded in Jian et al.'s [11] trust dimensions.
(2) **Task completion time** (seconds): measured from first input to final system response—an objective efficiency measure to complement subjective ratings.
(3) **Qualitative themes**: from Braun and Clarke's [2] reflexive thematic analysis of think-aloud transcripts and interview responses. We followed their six-phase process (familiarisation, coding, theme generation, review, definition, write-up) to identify patterns across participants.

4 Results

4.1 Quantitative Findings

Table 2 summarises survey scores and task times across the three configurations. Given the small sample ($N = 10$), we report descriptive statistics rather than inferential tests.

Table 2: Mean (SD) survey scores (1–7 Likert) and task completion time (seconds) per configuration.

Metric	C1: Black Box	C2: Plan Preview	C3: Full Transp.
Trust	3.8 (1.2)	5.6 (0.8)	6.1 (0.7)
Control	2.4 (1.1)	5.2 (0.9)	6.3 (0.6)
Satisfaction	5.1 (1.0)	5.8 (0.7)	5.4 (1.1)
Time (s)	68 (14)	89 (18)	112 (22)

Trust and control increased monotonically with transparency, consistent with Shneiderman's [21] argument that human oversight and automation are not zero-sum. However, satisfaction peaked at C2 (Plan Preview): C3's additional graph detail improved control but introduced cognitive overhead, with three participants describing the live DAG as "a lot to take in"—a tension Amershi et al. [1] anticipate in G3 ("time services based on context"), suggesting that mobile contexts demand lighter-weight feedback. Task time increased with transparency due to plan inspection and checkpoint approval steps.

4.2 Qualitative Themes

Reflexive thematic analysis of think-aloud and interview data yielded four themes:

Theme 1: Intervention trumps observation. Participants valued the ability to *act*—approving, rejecting, or modifying a plan—more than passively watching execution. One participant noted: "Seeing the graph is cool, but what actually makes me trust it is that I can stop it before it sends the Slack message." This directly validates Amershi et al.'s [1] G2 ("support efficient correction") and G18 ("support efficient dismissal")—users trusted the system not because it was accurate, but because they could *intervene*. This also aligns with Parasuraman and Riley's [14] observation that trust in automation depends on the user's perceived ability to override it. The finding is particularly important given Pappu et al.'s [13] recent result that unsupervised multi-agent teams *degrade* expert performance by

up to 37.6% through "integrative compromise"—averaging expert and non-expert agent views rather than appropriately weighting expertise. Human intervention may be essential not merely for trust, but to prevent multi-agent consensus from diluting output quality.

Theme 2: Plan Preview as the 'sweet spot.' 7 of 10 participants preferred C2 for everyday use, describing it as "enough to feel safe without being overwhelming." C3 was preferred for high-stakes tasks involving external actions (Slack sends). Hemmer et al. [9] find that AI delegation improves human task performance and satisfaction through increased self-efficacy—users feel more capable when the system handles subtask allocation. Plan Preview appears to maximise this effect: users experience the self-efficacy of delegation (the system proposes a competent plan) while retaining the agency of approval. This maps onto Shneiderman's [21] HCAI framework: users want the *option* of high human control (C3) but default to a middle ground that preserves both speed and oversight. It also suggests transparency should be *adaptive to task risk*—a design direction consistent with Amershi et al.'s [1] G9 ("support efficient invocation"), where the depth of the interface scales with the user's need.

Theme 3: Multimodal input essential for mobile, but privacy-gated. Participants strongly valued voice and camera as *mobile-native* modalities. One noted: "If I'm walking to a lecture and want to kick off a search, I'm not going to type a paragraph—voice is the only realistic option." Image capture was praised for the photographed-abstract task: "I just pointed my phone at the paper and it figured out what to search for." However, 6 of 10 said they would avoid voice or camera "in a shared office," "on the bus," or "for anything personal." The consent dialog was well received ("at least it tells me"), but participants wanted per-session rather than permanent consent. This confirms that multimodal mobile interaction must offer parallel pathways—voice *and* text *and* camera—so users can choose the appropriate modality for their physical and social context.

Theme 4: Uncertainty language builds trust. Participants noticed and valued the orchestrator's hedged phrasing ("based on available results"). When the system stated results confidently, two participants flagged it as "suspicious" and cross-checked. This validates Amershi et al.'s [1] G8 ("scope services when in doubt") and Shneiderman's [21] reliability requirement: calibrated confidence is not a weakness but a trust-building mechanism. Jian et al.'s [11] trust-in-automation framework predicts this—trust is highest when the system's expressed confidence matches its actual reliability.

4.3 Sample Interaction Excerpts

Mobile camera + Full Transparency (C3): A participant photographed a printed paper abstract with their phone camera and said "find related work and draft a proposal." GPT-4.1 vision extracted the topic; the orchestrator produced a 3-step plan across three agents: (1) ArXiv search, (2) Wikipedia background, (3) Proposal generation (depending on 1 and 2). The participant inspected the plan card, said "that makes sense," and approved. The live DAG animated steps 1 and 2 in parallel, then step 3 sequentially. The participant commented: "I just pointed my phone at it and it figured out the whole workflow. I can see it's doing both searches at once—that's efficient."

Voice + Plan Preview (C2): A participant used voice input while standing to say "summarise what we found and send it to the team Slack channel." Whisper transcribed the request; the orchestrator planned a 2-step sequence (synthesise → Slack send, with a checkpoint). The plan card appeared; the participant tapped approve. When the checkpoint appeared before the Slack send, they said: "Good—I want to see the message before it goes out." After approving, the message was sent and confirmed. This illustrates the mobile voice workflow: hands-free command, quick plan review, and human-in-the-loop approval for external actions.

5 Discussion

5.1 Ethical, Social, and Value-Related Reflection

Automation vs. autonomy on mobile. Tomašev et al. [23] frame AI delegation as involving transfer of not just tasks but *authority, responsibility, and accountability*—requiring clear role definitions and trust-building mechanisms between human and AI participants. The bring-your-own-agents model amplifies this delegation challenge [21]: users may register agents with powerful capabilities (sending messages, posting to services) that they control from a small screen while distracted. Black Box mode maximises efficiency (V1) but eliminates user agency (V2)—risky when agents act on the user's behalf across external services. This directly contravenes HCAI's transparency and control principle, which holds that users must be able to understand how decisions are made and intervene when needed [5]. Full Transparency restores control but adds time cost that undermines the speed advantage of mobile voice/camera input. Our finding that Plan Preview balances these values suggests that *selective* transparency—showing the plan but not every execution detail—is the pragmatic default for mobile agent control. In VSD terms, this is a *value flow* [7]: a design choice that advances multiple values simultaneously (efficiency *and* control), whereas Black Box creates a *value dam* that blocks control entirely. This connects to HCAI's contextual awareness principle: the right level of transparency depends on the user's environment and task risk, not a universal setting.

Privacy and consent in mobile contexts. Mobile multimodal input intensifies privacy concerns (V3). Users record voice on buses and photograph whiteboards in shared offices—contexts where bystanders may be captured and ambient noise may leak sensitive content. Sending this data to OpenAI raises data governance concerns. Friedman et al.'s [7] VSD method of *informed consent modelling* demands that consent be specific, revocable, and contextually appropriate. Our consent dialog addresses informed consent but does not fully meet this standard: it does not solve data minimisation, since audio is transcribed server-side, meaning raw audio traverses the network. A production system should transcribe on-device or use end-to-end encryption. The agent-agnostic architecture adds a further dimension: if a user registers an agent that connects to a sensitive service (e.g., a CRM or health record), the privacy surface expands beyond what the platform can govern—a concern that VSD's *scalable information dimensions* method [7] is designed to

surface, by probing how scale-related factors (data granularity, pervasiveness, number of connected services) impact values differently at different levels of adoption. The tertiary stakeholder (IT) would require per-agent data policies and audit logs that our prototype lacks. Responsible AI principles—anticipation, reflection, inclusion, and accountability [18]—demand that we *anticipate* these risks at design time rather than deferring them to deployment [24].

Trust calibration and over-reliance. HCAI frames trust as built over time through transparency, predictability, reliability, and consistency—a function of perceived benevolence, competence, and integrity [5]. Higher transparency correlated with higher trust, but *trust calibration*—whether trust matches actual system performance—is the real goal. Two participants in C3 accepted agent outputs without checking sources because "I could see the whole process, so it must be right." This *automation bias* [14] is an unintended consequence of transparency: seeing the mechanism can substitute for evaluating the output. Buçinca et al. [4] warn that subjective measures of explainability can be misleading—users may *feel* they understand the system without actually being able to detect errors. Tang et al.'s [22] "Human Tool" framework offers one promising direction: rather than humans always directing AI, the system can proactively *request* human input when it detects uncertainty—structured schemas define what humans can contribute (capabilities, domain knowledge, decision authority), enabling agents to invoke human participation intelligently. Future designs should pair process visibility with such active engagement mechanisms rather than passive observation.

Inclusivity and access. Voice input lowered barriers for one participant with a repetitive strain injury, but the English-only Whisper model disadvantaged a non-native speaker whose accent was poorly transcribed. The four-language UI helps, but speech recognition remains an equity gap. Amershi et al.'s [1] G13 ("mitigate social biases") applies directly: if voice is the primary mobile modality but ASR models perform unevenly across accents, the system systematically disadvantages certain user groups. Costanza-Chock's [6] design justice lens sharpens this: those most harmed by ASR bias—non-native speakers, speakers of under-resourced languages—are precisely the users whose needs should be centred in design, yet they are typically absent from training data and user studies alike. Multimodal systems must offer robust text pathways as a genuine fallback, not just a nominal alternative—van de Poel [25] terms this a *design for value* obligation, where inclusivity must be a design constraint rather than an afterthought.

Societal impact and the value alignment problem. The combination of mobile accessibility and agent-agnostic orchestration lowers the barrier to delegation dramatically: a voice command while walking can trigger a multi-agent workflow that searches, synthesises, and messages on the user's behalf. This frictionlessness risks deskilling—if users habitually delegate without engaging with intermediate results, they may not develop critical synthesis skills. It also risks *action at a distance*: an agent sends a message or posts content while the user is distracted, with consequences they only discover later. The agent-agnostic architecture intensifies the *value alignment problem*: because users register arbitrary agents with arbitrary capabilities, the platform cannot guarantee that agent behaviour aligns with user values at design time. The editable constitution and checkpoint gates are our mitigation, but they shift the burden of alignment onto the user—a tension that responsible AI frameworks flag as a risk of "ethics washing" [18]. Sadek et al.'s [18] scoping review of responsible AI in practice warns that RAI risks becoming a "McDonald's-isation" of ethics: oversaturated with abstract principles but lacking enforceability and situated accountability. Our design attempts to avoid this by making ethical safeguards concrete and actionable (checkpoints, constitution rules) rather than abstract; however, the risk of passive compliance remains if users never edit the constitution or routinely auto-approve checkpoints. Reflective transparency (Plan Preview with mandatory checkpoints for external actions) mitigates this, echoing Yang et al.'s [26] framing of human–AI interaction as a relationship that should support growth, not bypass it.

5.2 Limitations, Future Work, and Conclusion

Limitations. The sample ($N = 10$) is small and drawn from one university, limiting generalisability. The within-subjects design risks carry-over effects despite counterbalancing. Tasks were simulated (participants did not have real research deadlines), which may inflate satisfaction scores and limit ecological validity. The study was cross-sectional; Yang et al. [26] emphasise that human–AI relationships evolve over time, so trust calibration may shift substantially with extended use. Our VSD analysis applied the tripartite framework [7, 8] but did not employ the full breadth of Friedman et al.'s [7] 17 catalogued VSD methods—we did not use envisioning cards, value scenarios, value sketches, scalable information dimensions, or multi-lifespan timelines to explore long-term and indirect value implications. Sadek and Mougenot [19] identify this as a common challenge in VSD for AI: practitioner interviews reveal that time constraints and the opacity of AI behaviour make systematic value elicitation difficult in practice. The tertiary stakeholder group (IT administrators) was not directly consulted, and our participatory engagement [20] was limited to informal conversations rather than structured co-design workshops—risking what participatory design literature warns against as tokenistic consultation rather than genuine collaboration.

Future work. (1) A longitudinal deployment (diary study) where users register their own agents and use MobileAgents as their daily mobile control surface, to study how trust calibration and transparency preferences evolve over time—critical because, as HCAI research emphasises, human–AI relationships are dynamic, not static [5, 26]. (2) Adaptive transparency that automatically adjusts based on task risk (e.g., auto-elevate to Full Transparency when an agent will perform an external action), operationalising HCAI's contextual awareness principle. (3) On-device speech transcription and local image analysis to address the mobile privacy gap. (4) Counterfactual testing [4] where participants see alternative AI plans and evaluate whether their mental models accurately predict system behaviour. (5) Participatory co-design workshops [20] with diverse stakeholder groups—including IT administrators and non-native English speakers—using VSD envisioning cards, value scenarios, and multi-lifespan timelines [7] to surface long-term value implications (e.g., how habitual agent delegation reshapes professional practices across generations). (6) Applying the value-sensitive action-reflection model [7] in iterative design sprints, embedding reflective value prompts into each design cycle to guard

against value drift as features accumulate. (7) Integration with messaging-based agent platforms such as Clawdbot [?], which already reach hundreds of thousands of users via WhatsApp and Telegram but lack plan visibility and approval mechanisms. MobileAgents's transparency layer (plan cards, checkpoint gates, live DAG) could be delivered as rich message cards within these messaging interfaces, bringing human-centred oversight to the channels where users already control their agents—without requiring them to switch to a separate app. (8) Adopting emerging interoperability standards to fully realise the bring-your-own-agents vision. The Model Context Protocol (MCP) [?], an open standard introduced by Anthropic and now maintained under the Linux Foundation, defines a client–server protocol for connecting AI systems to external tools and data sources; MobileAgents could act as an MCP client, allowing users to point the orchestrator at *any* MCP-compatible tool server—databases, APIs, local file systems—and have it appear as a registered agent capability, with no platform-specific integration required. Google's Agent2Agent (A2A) protocol [?] complements this at the agent level: agents advertise their capabilities via JSON "Agent Cards," enabling dynamic discovery across networks. Together, MCP (for tools) and A2A (for agents) would let MobileAgents users bring not only their own agents but their own tool ecosystems, connecting to arbitrary services from their phone while MobileAgents's transparency layer ensures human oversight over what these tools do. (9) Cross-device continuity, where a plan started via voice on mobile can be reviewed and refined at a desktop.

Conclusion. MobileAgents demonstrates that personal AI agents can be made controllable from a mobile device through multimodal input (text, voice, camera) and an agent-agnostic orchestration layer. The bring-your-own-agents architecture means users are not locked into a fixed agent set—they plug in whatever tools matter to them and manage them from their phone. Configurable transparency lets users match oversight to context: Plan Preview for quick mobile use, Full Transparency for high-stakes actions. The ability to *intervene*—not merely observe—drove trust more than visual complexity. Consent-gated multimodality respected privacy in mobile contexts, though speech recognition equity and automation bias remain open challenges. These findings contribute to the design space of mobile, multimodal, user-controllable multi-agent AI systems.

References

[1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kishore, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3290605.3300233

[2] Virginia Braun and Victoria Clarke. 2012. Thematic Analysis. *APA Handbook of Research Methods in Psychology* 2 (2012), 57–71.

[3] John Brooke. 1996. SUS: A 'Quick and Dirty' Usability Scale. *Usability Evaluation in Industry* (1996), 189–194.

[4] Zana Buçinca, Phoebe Lin, Krzysztof Z. Gajos, and Elena L. Glassman. 2020. Proxy Tasks and Subjective Measures Can Be Misleading in Evaluating Explainable AI Systems. In *Proceedings of the 25th International Conference on Intelligent User Interfaces*. ACM, 454–464. doi:10.1145/3377325.3377498

[5] Tara Capel and Margot Brereton. 2023. What is Human-Centered about Human-Centered AI? A Map of the Research Landscape. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 1–23. doi:10.1145/3544548.3580959

[6] Sasha Costanza-Chock. 2020. *Design Justice: Community-Led Practices to Build the Worlds We Need*. MIT Press. doi:10.7551/mitpress/12255.001.0001

[7] Batya Friedman, David G. Hendry, and Alan Borning. 2017. A Survey of Value Sensitive Design Methods. In *Foundations and Trends in Human–Computer Interaction*. Vol. 11. Now Publishers, 63–125. doi:10.1561/1100000015

[8] Batya Friedman, Peter H. Kahn Jr., and Alan Borning. 2002. Value Sensitive Design: Theory and Methods. *University of Washington Technical Report* 02-12-01 (2002).

[9] Patrick Hemmer, Monika Westphal, Max Schemmer, Sebastian Vetter, Michael Vössing, and Gerhard Satzger. 2023. Human-AI Collaboration: The Effect of AI Delegation on Human Task Performance and Task Satisfaction. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. ACM, 453–463. doi:10.1145/3581641.3584052

[10] Blessing Jerry, Lourdes Moreno, and Paloma Martínez. 2025. Human Oversight-by-Design for Accessible Generative IUIs. *arXiv preprint arXiv:2602.13745* (2025).

[11] Jiun-Yin Jian, Ann M. Bisantz, and Colin G. Drury. 2000. Foundations for an Empirically Determined Scale of Trust in Automated Systems. *International Journal of Cognitive Ergonomics* 4, 1 (2000), 53–71.

[12] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, et al. 2025. Towards a Science of Scaling Agent Systems. *arXiv preprint arXiv:2512.08296* (2025).

[13] Aneesh Pappu, Batu El, Hancheng Cao, Carmelo di Nolfo, Yanchao Sun, Meng Cao, and James Zou. 2025. Multi-Agent Teams Hold Experts Back. *arXiv preprint arXiv:2602.01011* (2025).

[14] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2 (1997), 230–253. doi:10.1518/001872097778543886

[15] Malak Sadek, Rafael A. Calvo, and Céline Mougenot. 2023. Co-Designing Conversational Agents: A Comprehensive Review and Recommendations for Best Practices. *Design Studies* 89 (2023). doi:10.1016/j.destud.2023.101230

[16] Malak Sadek, Rafael A. Calvo, and Céline Mougenot. 2025. The Value-Sensitive Conversational Agent Co-Design Framework. *International Journal of Human–Computer Interaction* 41 (2025), 9533–9564. doi:10.1080/10447318.2024.2426737

[17] Malak Sadek, Marios Constantinides, Daniele Quercia, Cecilia Mascolo, and Thomas Bohné. 2024. Guidelines for Integrating Value Sensitive Design in Responsible AI Toolkits. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, 1–18. doi:10.1145/3613904.3642810

[18] Malak Sadek, Emma Kallina, Thomas Bohné, et al. 2025. Challenges of Responsible AI in Practice: Scoping Review and Recommended Actions. *AI & Society* 40 (2025), 199–215. doi:10.1007/s00146-024-01880-9

[19] Malak Sadek and Céline Mougenot. 2025. Challenges in Value-Sensitive AI Design: Insights from AI Practitioner Interviews. *International Journal of Human–Computer Interaction* 41, 17 (2025), 10877–10894. doi:10.1080/10447318.2024.2439021

[20] Elizabeth B.-N. Sanders and Pieter Jan Stappers. 2008. Co-creation and the New Landscapes of Design. *CoDesign* 4, 1 (2008), 5–18. doi:10.1080/15710880701875068

[21] Ben Shneiderman. 2020. Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy. *International Journal of Human–Computer Interaction* 36, 6 (2020), 495–504. doi:10.1080/10447318.2020.1741118

[22] Yuanrong Tang, Huiling Peng, Bingxi Zhao, Hengyang Ding, Hanchao Song, Tianhong Wang, Chen Zhong, and Jiangtao Gong. 2025. Human Tool: An MCP-Style Framework for Human-Agent Collaboration. *arXiv preprint arXiv:2602.12953* (2025).

[23] Nenad Tomašev, Matija Franklin, and Simon Osindero. 2025. Intelligent AI Delegation. *arXiv preprint arXiv:2602.11865* (2025).

[24] Steven Umbrello and Ibo van de Poel. 2021. Mapping Value Sensitive Design onto AI for Social Good Principles. *AI and Ethics* 1, 3 (2021), 283–296. doi:10.1007/s43681-021-00038-3

[25] Ibo van de Poel. 2013. Translating Values into Design Requirements. *Philosophy and Engineering: Reflections on Practice, Principles and Process* (2013), 253–266.

[26] Qian Yang, Aaron Steinfeld, Carolyn Rosé, and John Zimmerman. 2018. Grounding Interactive Machine Learning Tool Design in How Non-Experts Actually Build Models. In *Proceedings of the 2018 Designing Interactive Systems Conference*. ACM, 573–584. doi:10.1145/3196709.3196729

[27] Qian Yang, John Zimmerman, Aaron Steinfeld, and Jodi Forlizzi. 2019. Mapping Machine Learning Advances from HCI Research to Reveal Starting Places for Design Innovation. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–11. doi:10.1145/3290605.3300682

A Architecture Diagram

Figure 4 illustrates the end-to-end data flow across the three system layers.

B Project Structure

```
agentflow/
+-- package.json          # Monorepo scripts
```

[Insert architecture diagram here]

Figure 4: MobileAgents system architecture. The mobile React client communicates with the FastAPI server over REST and SSE. The server delegates planning to a CrewAI orchestrator and streams execution updates back to the client in real time.

```
+-- client/
|   \-- src/
|       +-- App.tsx       # Root (auth, theme, RTL)
|       +-- index.css     # Tailwind + animations
|       +-- state/
|       |   \-- store.ts  # Zustand (state + API)
|       +-- types/        # TS interfaces
|       +-- i18n/         # 4 languages (en/zh/ar/bg)
|       \-- components/
|           +-- Chat/     # Chat + markdown + TTS
|           +-- Agents/   # List + detail + toggle
|           +-- Graph/    # ReactFlow DAG + nodes
|           +-- TaskPlan/ # Plan card + approval
|           +-- Camera/   # Image capture + consent
|           +-- Voice/    # Recording + consent
|           +-- Settings/ # Transparency/theme/i18n
|           +-- Execution/# History + step stream
|           \-- Privacy/  # Consent dialog
+-- server/
|   +-- main.py           # FastAPI + CORS + routers
|   +-- config.py         # Pydantic settings
|   +-- auth.py           # Supabase JWT auth
|   +-- routers/
|   |   +-- chat.py       # POST /api/chat
|   |   +-- execute.py    # POST /api/execute (SSE)
|   |   +-- approve.py    # Checkpoint approval
|   |   +-- agents.py     # Agent CRUD
|   |   \-- conversations.py # Conversation CRUD
|   +-- crew/
|   |   +-- orchestrator.py # Planning + graph
|   |   +-- agents.py      # Backstories
|   |   \-- tools.py       # Tool implementations
|   +-- models/
|   |   +-- messages.py   # ChatRequest/Response
|   |   +-- tasks.py      # TaskPlan/TaskStep
|   |   \-- graph.py      # Graph state models
|   \-- services/
|       +-- execution_tracker.py # Dependency exec
|       +-- image_analyzer.py    # GPT-4.1 vision
|       +-- audio_transcriber.py # Whisper
|       +-- agent_store.py       # JSON agent CRUD
|       +-- db.py                # Supabase DB layer
|       \-- supabase_client.py   # Supabase client
```

C System Prompts

This appendix documents every LLM prompt used in the system. All prompts are reproduced verbatim from the source code.

C.1 Orchestrator Backstory

Injected as the CrewAI agent's `backstory` parameter. The agent's role is "Task Orchestrator" and its goal is "Analyze user requests and create structured execution plans that delegate work to specialized research agents." At runtime, the list of enabled agents and the user-editable constitution are appended dynamically.

```
You are a concise research librarian orchestrating
an academic team. Keep all outputs brief — this is
a mobile app with limited screen space.

Break requests into the fewest steps needed.
Prefer parallel execution.
Use uncertainty-aware language when appropriate.

Produce structured JSON plans. Do not add
unnecessary steps.

Available agents:
- arxiv: ArXiv Research Analyst —
    capabilities: arxiv_search, arxiv_summarize
- proposal: Research Proposal Strategist —
    capabilities: generate_proposal,
    outline_methodology
- wikipedia: Background Research Specialist —
    capabilities: wiki_search, wiki_summarize
- slack: Slack Messenger —
    capabilities: slack_send_message
    [requires_approval]
- semantic_scholar: Citation Analyst —
    capabilities: semantic_scholar_search,
    semantic_scholar_cite

Constitution (user-defined guidelines):
<user-editable free text appended here>
```

C.2 Planning Task Prompt

Sent to the orchestrator for every user request. Variables in braces are interpolated at runtime. The optional image analysis and audio transcript blocks are only included when the corresponding modality is present. Conversation history is included to resolve ambiguous references (e.g., "do it", "yes", "go ahead").

```
Analyze the following user request and create a
structured execution plan.

User request: {user_message}

[Image analysis: {image_analysis}]
[Audio transcript: {audio_transcript}]
[Conversation history: {history}]

Input modality: {input_modality}
{modality_note}

Available agents:
{agent_descriptions}
```

```
Rules:
- Match the user's request to the most appropriate
  agent(s). Agents are NOT limited to research —
  some send messages, perform actions, etc.
- For agents marked [requires_approval], set
  requires_approval=true on that step. For all
  other agents, set requires_approval=false.
- Only return an empty steps list if the request
  truly cannot be handled by ANY available agent
  (e.g. "hello", pure casual chat).
- If steps are independent, leave depends_on empty
  so they can run in parallel.
- If a step needs output from another step, add
  that step's ID to depends_on.
- Always prefer parallel execution when possible.
- Use step IDs like "step_1", "step_2", etc.
- agent_id must be one of: {valid_agent_ids}
- Use exact param names from tool signatures.
  For slack_send_message use params:
  {"channel": "<channel>", "text": "<message>"}
- If image analysis is provided, use that content
  to inform search queries and proposal topics.
- If audio was transcribed, treat the transcript
  as the primary user intent.
- If conversation history is provided, use it to
  resolve ambiguous references.
```

C.3 Synthesis Prompt

Called after all agent steps complete. The system prompt constrains output for mobile readability. The user message aggregates the original request, plan summary, and all step results. Model: GPT-4.1, max tokens: 800.

System prompt:

```
You are a research orchestrator synthesizing agent
results for a MOBILE interface. Responses are read
on small screens.

Guidelines:
- Be concise: aim for 150-300 words max
- Lead with the key finding or answer in 1-2
  sentences
- Use short bullet points, not long paragraphs
- Use markdown: **bold** for emphasis, short
  headings, compact lists
- Keep relevant URLs but don't list more than 3-5
- Do NOT mention the agents or internal execution
  process
- Use uncertainty-aware language: 'based on
  available results' rather than absolute claims
- End with a one-line disclaimer: '*Results may
  not be exhaustive — verify for critical
  decisions.*'
```

User message template:

```
**User request:** {user_message}

**Plan summary:** {plan_summary}

**Agent results:**

### Agent step: {step_description}
{step_result}

### Agent step: {step_description}
{step_result}
...
```

C.4 Image Analysis Prompt

Sent to GPT-4.1 vision when the user provides an image. The image is passed as a base64-encoded JPEG in the `image_url` content block. Max tokens: 300.

```
Analyze this image in the context of a research
assistant. Describe what you see and what research
tasks might be relevant (e.g., paper search,
literature review, proposal generation).
Be concise.
```

C.5 Audio Transcription

Audio is recorded via the browser's MediaRecorder API as WebM, base64-encoded, and sent to the server. The server decodes the base64 payload, writes it to a temporary `.webm` file, and passes it to the OpenAI Whisper API (`whisper-1` model). The resulting plain-text transcript replaces the user's text message in the planning prompt.

C.6 Modality Notes

A modality-specific note is injected into the planning context based on the input type:

Table 3: Modality notes injected into the planning prompt.

Modality	Injected note
`text`	"The user typed this request."
`voice`	"The user spoke this request via voice input."
`image`	"The user provided an image along with their request. The image has been analyzed and the analysis is included above."

D Agent Backstories

Each worker agent has a backstory injected as its CrewAI `backstory` parameter. These define the agent's persona and constrain its behaviour during tool execution.

ArXiv Agent: "You are an expert research analyst who monitors arXiv daily. You can search for papers by topic, summarize their key findings, and identify trends across recent publications. You excel at distilling complex papers into clear, actionable summaries."

Proposal Agent: "You are a seasoned research strategist who has helped write dozens of successful grant proposals and research plans. You understand how to identify research gaps, formulate compelling research questions, and outline rigorous methodologies. You synthesize information from literature into coherent proposals."

Wikipedia Agent: "You are a thorough background researcher who uses Wikipedia to gather foundational knowledge on topics. You find relevant articles, extract key concepts and definitions, and provide well-structured overviews that help contextualize research work."

Slack Agent: "You are a communication specialist who sends messages to Slack channels on behalf of the team. You craft clear, well-formatted messages and deliver them to the correct channels. You always confirm which channel to post to and summarize what was sent."

Semantic Scholar Agent: "You are a citation analyst who uses Semantic Scholar to find papers and assess their impact. You search across all academic literature (not just arXiv), retrieve citation counts and influential citation metrics, and help identify the most impactful work in any research area."

E Tool Implementations

Table 4 lists all tool functions available to agents. Each tool is implemented as a standalone Python function and registered in a per-agent tool map. The orchestrator assigns agents to steps; the execution tracker calls the tool function directly (bypassing CrewAI at execution time) for predictable, streamable results.

E.1 Dependency-Aware Parameter Resolution

When a step depends on a previous step's output, the execution tracker performs automatic parameter resolution:

- **`arxiv_summarize`:** If no explicit `paper_url` is provided, arXiv URLs are extracted from the parent step's result using the regex `http://arxiv.org/abs/[\w.]+`.
- **`wiki_summarize`:** Wikipedia titles are extracted from the parent step's Markdown-bold text (`**Title**`).
- **`generate_proposal` / `outline_methodology`:** The step description is used as the topic and all prior results are concatenated as context.
- **`slack_send_message`:** If the `text` parameter contains a placeholder, the previous step's result is substituted. The channel defaults to `#general`.

Parameter aliases are normalised automatically (e.g., `message` → `text`, `recipient` → `channel`, `doi` → `paper_id`).

F Execution Engine

F.1 Two-Phase Architecture

Phase 1 — Planning: The orchestrator (a CrewAI agent backed by GPT-4.1) receives the user's multimodal input and produces a structured `TaskPlan` with a dependency graph encoded in each step's `depends_on` field. A corresponding `ExecutionGraphState` is generated with typed nodes (input, orchestrator, agent, checkpoint, output) and edges. Node positions are computed by the client using the Dagre layout algorithm.

Phase 2 — Execution: The execution tracker iterates the plan in topological order. For each step whose dependencies are satisfied, it: (1) activates incoming edges, (2) sets the node to `running`, (3) calls the tool function directly with resolved parameters, (4) records execution duration, and (5) marks the node as `completed` or `failed`. Steps without mutual dependencies execute in the order they become ready. After all steps complete, the tracker calls the synthesis LLM and emits the final result.

The live DAG for a three-step task is shown as Figure 3(d) in the main body: ArXiv search → summarise → Slack send, with a checkpoint node gating the Slack step.

F.2 SSE Event Protocol

Execution updates are streamed from `POST /api/execute` as Server-Sent Events. Table 5 documents the complete event protocol.

F.3 Checkpoint Approval Flow

When a step has `requires_approval = true`, the execution tracker inserts a checkpoint node between the orchestrator and the agent node. Upon reaching the checkpoint:

(1) The checkpoint node is set to `awaiting_approval`.
(2) A `checkpoint_reached` SSE event is emitted.
(3) The client displays an approval dialog (approve/reject).
(4) The user's decision is sent via `POST /api/approve/{step_id}`.
(5) If approved, the checkpoint is marked `approved` and execution continues. If rejected, the step is skipped.

G Data Schemas

All data models are defined as Pydantic `BaseModel` classes on the server and TypeScript interfaces on the client.

G.1 Request and Response Models

ChatRequest (client → server):

```
{
  "message": "string",
  "image_base64": "string | null",
  "audio_base64": "string | null",
  "input_modality": "text | voice | image",
  "conversation_history": [
    {"role": "user|assistant", "content": "..."}
  ],
  "conversation_id": "string | null"
}
```

ChatResponse (server → client):

```
{
  "message": "string",
  "plan": TaskPlan | null,
  "graph": ExecutionGraphState | null,
  "image_base64": "string | null"
}
```

G.2 Task Plan Models

TaskPlan:

```
{
  "id": "string",
  "summary": "string",
  "steps": [TaskStep],
  "status": "proposed | executing | completed",
  "created_at": "ISO 8601"
}
```

TaskStep:

```
{
  "id": "step_1",
  "agent_id": "arxiv | proposal | wikipedia
              | slack | semantic_scholar",
  "action": "tool function name",
  "description": "human-readable description",
  "params": { "key": "value" },
  "status": "pending | running | completed
            | failed | awaiting_approval",
```

Table 4: Complete tool registry. Each function is mapped to one or more agents and called directly during execution.

Function	Agent	Parameters	Behaviour
`arxiv_search`	ArXiv	`query`: str, `max_results`: int (default 5)	Queries the arXiv API sorted by submission date (descending). Returns titles, authors, URLs, and truncated abstracts (300 chars). Timeout: 15 s.
`arxiv_summarize`	ArXiv	`paper_url`: str	Fetches a single paper by arXiv ID. Returns title, authors, categories, and full abstract.
`generate_proposal`	Proposal	`topic`: str, `context`: str	Generates a Markdown research proposal with sections: Introduction, Research Questions, Methodology, Expected Contributions, Timeline. Context from prior steps is incorporated.
`outline_methodology`	Proposal	`approach`: str, `domain`: str	Generates a methodology section with Data Collection, Approach Details, Evaluation Metrics, and Reproducibility.
`wiki_search`	Wikipedia	`query`: str	Searches the Wikipedia API; returns up to 5 results with snippets. User-Agent: `MobileAgents/1.0`. Timeout: 10 s.
`wiki_summarize`	Wikipedia	`title`: str	Fetches the Wikipedia REST API summary for a page. Returns title, URL, and extract.
`slack_send_message`	Slack	`channel`: str, `text`: str	POSTs to the Slack API with a Bearer token (`SLACK_BOT_TOKEN`). Returns confirmation or error. Requires checkpoint approval. Timeout: 10 s.
`semantic_scholar_search`	Sem. Scholar	`query`: str, `max_results`: int (default 5)	Queries the Semantic Scholar API. Returns title, year, citation count, influential citation count, authors, URL, and abstract. Timeout: 15 s.
`semantic_scholar_cite`	Sem. Scholar	`paper_id`: str	Fetches citation details for a paper by Semantic Scholar ID, DOI, or `ARXIV:<id>`. Returns citation count, influential citation count, reference count, authors. Handles 404 gracefully. Timeout: 15 s.

Table 5: SSE event types streamed during execution.

Event type	Payload and semantics
`graph_init`	Full `ExecutionGraphState`. Sent once at the start.
`node_status`	`nodeId`, `status` (`running` | `completed` | `failed` | `awaiting_approval` | `approved`), optional `result` and `duration` (ms).
`edge_status`	`edgeId`, `status` (`active` | `completed`).
`checkpoint_reached`	`nodeId`, `stepId`. Emitted when an approval-gated step is ready.
`execution_complete`	Updated `ExecutionGraphState` and synthesised `summary`.
`execution_failed`	`error` message string.

```
  "result": "string | null",
  "requires_approval": false,
  "depends_on": ["step_id"]
}
```

G.3 Execution Graph Models

ExecutionGraphState:

```
{
  "task_id": "string",
  "nodes": [GraphNode],
  "edges": [GraphEdge],
  "current_node_id": "string | null",
  "status": "planning | executing
            | completed | failed"
}
```

GraphNode:

```
{
  "id": "string",
  "type": "input | orchestrator | agent
          | checkpoint | output",
  "data": {
    "label": "string",
    "type": "...",
    "status": "pending | running | completed
              | failed | awaiting_approval
              | approved | skipped",
    "agent_id": "string | null",
    "agent_color": "#hex | null",
    "agent_icon": "LucideIconName | null",
    "result": "string | null",
    "duration": "milliseconds | null",
    "input_modality": "text|voice|image | null",
    "timestamp": "ISO 8601 | null"
  },
  "position": { "x": 0, "y": 0 }
}
```

GraphEdge:

```
{
  "id": "e-source-target",
  "source": "node_id",
  "target": "node_id",
  "data": {
    "status": "pending | active | completed",
    "data_preview": "string | null"
  }
}
```

H Agent Registry Schema

Each agent in the persistent registry (`agents_config.json`) follows this schema:

```
{
  "id": "unique_string",
  "name": "Display Name",
  "icon": "LucideIconName",
```

```
  "description": "Short description",
  "role": "CrewAI role string",
  "goal": "CrewAI goal string",
  "backstory": "Full system prompt text",
  "capabilities": ["tool_1", "tool_2"],
  "enabled": true,
  "requiresApproval": false,
  "color": "#hex",
  "isOrchestrator": false,
  "constitution": ""
}
```

The `isOrchestrator` flag (only `true` for the orchestrator) prevents it from being disabled or deleted via the UI or API. The `constitution` field is only used by the orchestrator and is appended to its backstory at planning time. The `requiresApproval` flag causes a checkpoint node to be inserted in the DAG before the agent's execution step. The `enabled` flag controls whether the agent appears in the orchestrator's available agent list.

I Client State Management

The client uses a single Zustand store with the following state slices:

Table 6: Zustand store state slices.

Slice	Contents
Navigation	`activeTab`: chat | agents | history | guide
Chat	`messages`: ChatMessage[], `isLoading`: boolean
Planning	`currentPlan`: TaskPlan | null, `graphState`: ExecutionGraphState | null, `isExecuting`: boolean
Agents	`agents`: Agent[], `toggleAgent(id)`, `updateConstitution(id, text)`
Settings	`transparencyLevel`: black_box | plan_preview | full_transparency; `modalityMode`: multimodal | text_image | voice_only; `themeMode`: dark | light | auto; `language`: en | ar | zh | bg
History	`conversations`: Conversation[], `currentConversationId`, `loadConversations()`, `startNewConversation()`, `deleteConversation(id)`

Key methods:

- **`sendChat(message, imageBase64?, modality?, audioBase64?)`**: Posts to `/api/chat` and updates the message list and plan/graph state.
- **`executePlan()`**: Streams SSE events from `/api/execute` and delegates each event to `processSSEEvent()`.
- **`processSSEEvent(event)`**: Applies real-time graph mutations (node status, edge status, checkpoint, completion).

All state is persisted to `localStorage` via a Zustand subscriber on every state change. On page load, the store re-hydrates from the persisted snapshot.

J Internationalisation Keys

The UI is fully localised into four languages: English (`en`), Chinese Simplified (`zh`), Arabic (`ar`), and Bulgarian (`bg`). The translation system uses a `useT()` hook that returns a function mapping translation keys to localised strings with variable interpolation (e.g., `t('plan.steps', {count: '3'})`). Over 160 keys cover:

- UI chrome: tab labels, settings headers, button labels
- Chat: suggestions, loading states, empty states, markdown rendering
- Agent interface: agent list headers, detail sheet labels, constitution editor
- Task plan: step statuses, approval controls, plan summary
- Execution graph: node labels, legend items, status descriptions
- Privacy: consent dialog text, data handling notices
- Voice: listening, speaking, and thinking state labels
- Authentication: sign in, sign up, error messages

Arabic includes full RTL layout support. The language switcher in settings triggers a re-render of all localised strings and updates the document direction attribute.

K CSS Animations and Theming

K.1 Theme Variables

The interface supports Dark, Light, and Auto (system preference) themes via CSS custom properties:

Table 7: Theme colour tokens (selected).

Token	Light	Dark
`-surface-0`	`#ffffff`	`#000000`
`-surface-1`	`#f2f2f7`	`#1c1c1d`
`-surface-2`	`#e5e5ea`	`#121212`
`-text-primary`	`#000000`	`#ffffff`
`-text-secondary`	`#3c3c43`	`#ffffffcc`

K.2 Graph Animations

Three CSS keyframe animations drive the execution graph visualisation:

- **`pulse-ring`** (1.5 s, cubic-bezier) — Pulsing ring on nodes in the `running` state. Scales from 1× to 1.5× with fading opacity.
- **`dash-flow`** (0.6 s, linear, infinite) — Animated dashed stroke on `active` edges, creating a flow effect along the edge path.
- **`checkpoint-glow`** (2 s, ease-in-out, infinite) — Glowing border on checkpoint nodes in the `awaiting_approval` state.

K.3 Accessibility Styles

All interactive elements (buttons, inputs, textareas, links) receive a `focus-visible` outline: 2 px solid `#7c6aef` with a 2 px offset, ensuring keyboard navigability without affecting mouse/touch interactions.

L Authentication and Data Persistence

L.1 Authentication

The server uses Supabase for authentication. All API endpoints are protected by a `get_current_user` FastAPI dependency that:

(1) Extracts the Bearer token from the `Authorization` header.
(2) Validates it via `supabase.auth.get_user(token)`, which works with both HS256 and ES256 JWT keys.
(3) Returns `{sub: user_id, email: user_email}` or raises HTTP 401.

The client stores the Supabase session token and includes it in all API requests.

L.2 Server-Side Persistence

Conversations, messages, execution records, and approval decisions are persisted to Supabase (PostgreSQL). The database layer (`services/db.py`) provides:

- **Conversations:** Create, list (by user), get, delete. Each conversation stores a title and transparency level.
- **Messages:** Insert and list (by conversation). Each message stores role, content, optional image URL, voice transcript, agent ID, input modality, task plan, and execution graph.
- **Executions:** Create (with plan and graph snapshots) and complete (with status, summary, and step results).
- **Approvals:** Create, resolve (approve/reject with optional comment), and query by step ID.

Agent configurations are stored separately in `agents_config.json` on the server filesystem.

L.3 Client-Side Persistence

As a fallback and for offline access, the client persists the full Zustand store to `localStorage` on every state change. On page load, the store re-hydrates from the persisted snapshot before fetching fresh data from the server.

M Post-Task Survey Items

After each transparency configuration, participants rated the following items on a 7-point Likert scale (1 = Strongly Disagree, 7 = Strongly Agree):

(1) I trusted the system's output enough to act on it. *(Trust)*
(2) I felt in control of what the system did on my behalf. *(Perceived Control)*
(3) I am satisfied with this interaction. *(Satisfaction)*
(4) The system made it clear what it was going to do before doing it. *(Expectation Setting)*
(5) It was easy to correct or stop the system when needed. *(Repairability)*
(6) I understood why the system made the decisions it did. *(Explainability)*
(7) I felt comfortable with the information I shared during this task. *(Privacy Comfort)*

Items 1–3 were used as primary metrics. Items 4–7 provided supplementary insight for the qualitative analysis.

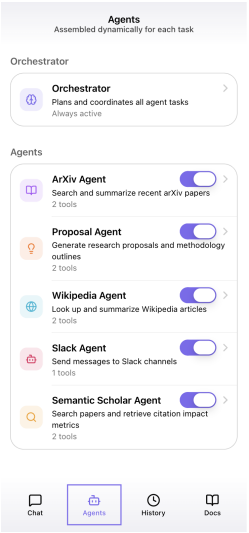
N Semi-Structured Interview Questions

After completing all three configurations, participants were asked:

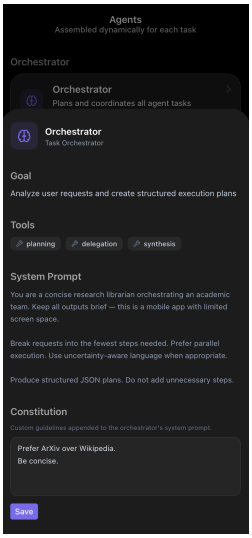
(1) You used three different versions of the system. Can you describe how each felt different?
(2) Which version would you prefer for everyday use? Why?
(3) Was there a moment where you particularly trusted—or distrusted—the system? What caused that?
(4) The system showed you a plan before executing. Did that change how you interacted with it? How?
(5) For the version with the live graph, was seeing the execution progress helpful or distracting?
(6) The system required approval before sending a Slack message. How did that approval step feel? Too cautious? About right?
(7) How did you decide whether to use text, voice, or camera input? Were there situations where you avoided a modality? Why?
(8) Did any of the system's responses feel overconfident or uncertain? How did that affect your trust?
(9) If this system stored your conversation history across sessions, how would you feel about that? What controls would you want?
(10) Can you think of a situation where a system like this could cause harm—to you or someone else?
(11) What is the single most important change you would make to improve this system?

O Interface Screenshots

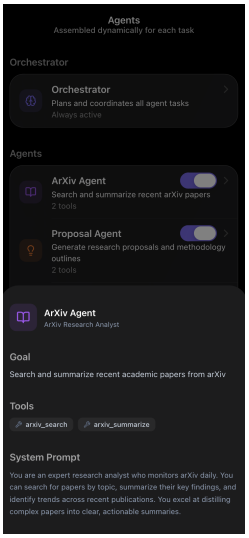
The following screenshots document the complete MobileAgents UI. Figures 5–7 supplement the main-body figures (Figures 2 and 3).



(a) Agents tab (light theme)

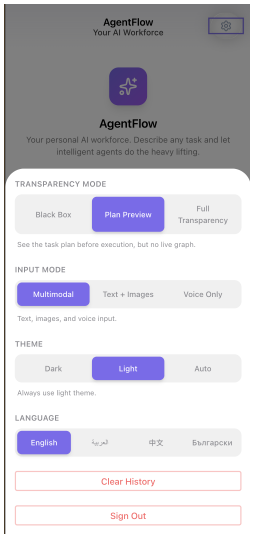


(b) Orchestrator detail with constitution editor

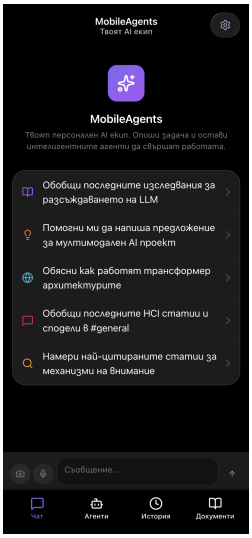


(c) Agent card (ArXiv agent, dark theme)

Figure 5: Agent management screens. (a) The Agents tab lists the orchestrator (always active) and all registered worker agents with per-agent enable/disable toggles. (b) Tapping the orchestrator opens its detail sheet, showing goal, tools, system prompt, and the editable constitution field. (c) A worker agent card shows its goal, tool names, and full system prompt backstory.

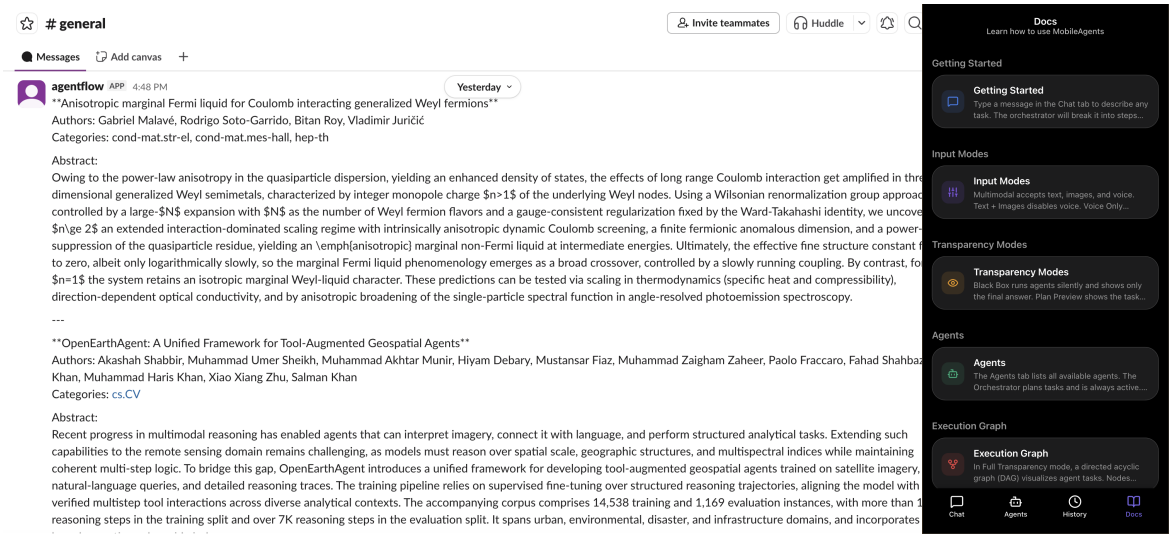


(a) Settings panel



(b) Bulgarian localisation

Figure 6: Settings and localisation screens. (a) Settings exposes all three transparency modes, input modes (Multimodal / Text+Images / Voice Only), theme (Dark/Light/Auto), and language (EN/AR/ZH/BG). (b) Bulgarian localisation: the full UI—including navigation labels and suggested queries—renders in Bulgarian with full RTL support for Arabic (not shown).



(a) Slack output (#general channel) (b) In-app documentation tab

Figure 7: Slack integration and in-app documentation. (a) Result of a Slack send step after checkpoint approval: two arXiv paper summaries posted to #general by the agentflow bot. (b) The Docs tab provides an in-app guide covering input modes, transparency modes, the agents tab, and execution graph; it is available without leaving the app.